SHARED GRAPH ARCHITECTURE

Proactive Sweep

On-Demand Context

→

Shared FleetGraph

ONE GRAPH, TWO TRIGGERS

FETCH | SIGNALS | REASONING | HITL

PUSH
Proactive findings

PULL
Context-aware answers

ACT
Safe draft actions

SHIPSHAPE

FleetGraph

Two Modes, One Shared Graph

ShipShape FleetGraph Feature Summary

PROACTIVE + ON-DEMAND | SHARED LANGGRAPH | HUMAN APPROVAL BOUNDARY

CONTEXT-AWARE SPRINT-RISK DETECTION, GROUNDED REASONING, AND SAFE ACTION PROPOSALS INSIDE SHIP

Define The Problem

WHAT

Sprint risk is easy to miss because teams still depend on manual checking across Ship views, activity history, and review state

WHY

The most important problems are often silence failures, such as no progress, no standup, or no follow-up, so by the time someone asks, the sprint is already drifting

HOW

Frame FleetGraph as a context-aware agent inside Ship that watches live project state, reasons over execution drift, and surfaces only meaningful findings

OUTCOME

We establish a strong product need for an agent that can both push insights proactively and answer questions on demand

What FleetGraph Is

WHAT

FleetGraph is one shared LangGraph system inside Ship with two modes: proactive and on-demand

WHY

The same reasoning should work whether the trigger is a timed sweep, a future event trigger, or a user opening a Ship view and asking for help

HOW

A supervisor-style graph routes into proactive or on-demand initialization, then reuses shared context, fetch, signal, reasoning, action, HITL, and fallback nodes

OUTCOME

We avoid duplicated logic, keep behavior consistent, and make both modes traceable through the same architecture

Agent Responsibility

WHAT

FleetGraph monitors sprint drift, stale work, low recent activity, and approval or review bottlenecks, then explains risk and proposes the next move

WHY

The MVP needed a narrow and defensible responsibility definition instead of a broad AI assistant with unclear boundaries

HOW

The responsibility, autonomy boundaries, notification order, use cases, and trigger model are defined in `FLEETGRAPH.md`

OUTCOME

The feature has a clear scope, a clear safety boundary, and a stronger story for both implementation and grading

Proactive Mode

WHAT

FleetGraph runs without a user present to detect active sprints that are drifting before anyone asks

WHY

A dashboard-only or chat-only assistant would miss time-based problems caused by silence, not just explicit Ship mutations

HOW

The MVP uses a hybrid trigger model with a manual proactive sweep route and an env-gated 5-minute worker, then persists findings, dedupes them, and delivers realtime notifications

OUTCOME

At least one proactive detection path is wired end-to-end against real Ship data

On-Demand Mode

WHAT

FleetGraph answers the MVP question, `Why is this sprint at risk?`, from the current Ship view

WHY

Users should not lose context or switch to a standalone chatbot to understand execution risk

HOW

The UI sends typed Active View Context with the current route, entity, tab, and project scope, and the graph resolves project or My Week views down to the current sprint before analysis

OUTCOME

FleetGraph gives page-aware answers directly inside Ship on week, project, and single-project My Week surfaces

Reasoning And Branching

WHAT

FleetGraph first derives deterministic sprint-risk signals and then produces grounded reasoning over the fetched evidence

WHY

The model should explain suspicious state, not invent facts or act as the first filter for every run

HOW

The graph fetches sprint data in parallel, derives signals like missing standup, no completed work, low recent activity, and changes requested, then uses optional model reasoning with deterministic fallback

OUTCOME

The graph produces visibly different paths for quiet runs, surfaced findings, and fallback cases while keeping the explanation grounded

Human-In-The-Loop Actions

WHAT

FleetGraph can draft follow-up or escalation comments, but it pauses before taking a consequential action

WHY

The assignment requires human approval before persistent mutations, and this is the right safety boundary for Ship

HOW

The graph proposes an action, interrupts for approve, dismiss, or snooze, records that decision in action memory, and only posts the sprint comment if the human approves

OUTCOME

The system is useful without becoming reckless, and it avoids repeatedly proposing the same action after approval, dismissal, or snooze

Observability And Evidence

WHAT

FleetGraph captures execution evidence, node history, and trace metadata for each run

WHY

This project is graded on graph behavior and branching, not just on whether a panel appears in the UI

HOW

Invoke and resume paths attach LangSmith callbacks, resolve shared trace links, record node-level telemetry, and use local evidence scripts to verify quiet and flagged runs

OUTCOME

Local MVP evidence is strong, with two shared LangSmith traces already captured, and the main remaining verification gap is public deployment

Feature Outcomes

WHAT

The feature delivers proactive sprint-risk detection, context-aware on-demand explanation, safe action proposals, and durable operational memory

WHY

The goal was not just to build a graph, but to reduce time-to-awareness, reduce ambiguity, and make the next action clearer for the team

HOW

We combined one shared graph, Active View Context, real Ship fetches, deterministic signals, grounded reasoning, HITL gating, finding persistence, and realtime delivery

OUTCOME

Teams get earlier detection of execution drift, faster sprint triage, less notification spam, and a safer path from insight to action

Next Enhancements

WHAT

The remaining work is final closeout plus post-MVP expansion: public deployment, latency proof, full test-case documentation, cost analysis, Claude-aligned AI integration, and broader trigger and surface coverage

WHY

These are the remaining requirements for full submission readiness and the next step from a locally proven MVP to a production-ready FleetGraph feature

HOW

Deploy FleetGraph routes publicly, prove `< 5 minute` detection latency with a timed run, add trace-linked test cases for each use case in `FLEETGRAPH.md`, complete development and production AI cost analysis, align the reasoning backend with the Claude API requirement, and later add high-signal mutation triggers plus wider issue, program, and dashboard coverage

OUTCOME

FleetGraph moves from a strong local MVP to a fully compliant, measurable, publicly verifiable, and expandable Ship intelligence layer